

pg-mcp

Product Specification

A local PostgreSQL MCP server with multi-database management, active connection gating, and a Tauri-based config UI.

VERSION	1.0
DATE	April 2026
AUTHOR	Grady
STATUS	Draft
RUNTIME	Tauri (Rust + Webview)
TRANSPORT	MCP over stdio

Table of Contents

- 1 Overview**
- 2 Architecture**
 - 2.1 System diagram
 - 2.2 Tauri application structure
 - 2.3 Config file format
- 3 Active Connection Model**
 - 3.1 Core principle
 - 3.2 Switching behavior
 - 3.3 What the agent sees
- 4 Safety Model**
 - 4.1 Read-only enforcement
 - 4.2 Write detection
 - 4.3 Connection isolation
- 5 MCP Tools**
 - 5.1 Tool reference
 - 5.2 Response format
 - 5.3 Error handling
- 6 Config UI**
 - 6.1 Active connection selector
 - 6.2 Connection cards
 - 6.3 Add/edit modal
 - 6.4 Agent setup panel
- 7 CLI Interface**
- 8 Visual Design**
 - 8.1 Color system
 - 8.2 Typography
 - 8.3 Component tokens
- 9 Distribution & Installation**
- 10 Future Considerations**

1. Overview

pg-mcp is a local desktop application that acts as a Model Context Protocol (MCP) server for PostgreSQL databases. It provides a visual UI to configure multiple database connections, a hard-gated active connection selector so the user controls exactly which database their AI agent can access, and a comprehensive set of MCP tools for schema exploration, query execution, and data inspection.

The application is built with Tauri, producing a lightweight native binary (~5MB) with a webview-based UI. It communicates with AI agents via MCP over stdio transport, making it compatible with Claude Desktop, Claude Code, Cursor, and any other MCP-compatible client.

Core principles

- **User-gated access** — The user selects one active database in the UI. The agent can only query that database. No exceptions.
- **Read-only by default** — Every new connection blocks write operations until explicitly enabled by the user.
- **Clear identification** — Every MCP tool response includes a banner showing the database name, host, and access mode.
- **Provider-agnostic** — Works with any MCP client. Not locked to a single AI provider.
- **Local-first** — Credentials never leave the user's machine. Config stored with restricted file permissions.

Target user: A developer working across multiple projects and databases who uses AI coding agents in their daily workflow and needs confidence that the agent is hitting the right database.

2. Architecture

2.1 System diagram

pg-mcp has three runtime surfaces that share a single config file:

Component	Runtime	Purpose
Tauri App	Native binary + webview	Config UI, active connection selector, system tray
MCP Server	Rust sidecar / spawned process	Handles stdio MCP transport, executes queries against active DB
CLI	Same binary, different entrypoint	pg-mcp activate, pg-mcp test, pg-mcp list

The Tauri app is the primary interface. When launched, it starts the webview UI and optionally places an icon in the system tray. The MCP server runs as a separate process spawned by the AI agent's MCP client (e.g., Claude Desktop runs `pg-mcp serve` via stdio). Both the UI and the MCP server read from the same config file, so changing the active connection in the UI is immediately reflected in the next MCP tool call.

2.2 Tauri application structure

Tauri applications consist of a Rust backend (for system access, file I/O, process management) and a web frontend rendered in the OS's native webview (WebKit on macOS, WebView2 on Windows, WebKitGTK on Linux). This architecture produces binaries in the 5-10MB range, compared to 150MB+ for Electron.

```
pg-mcp/  
src-tauri/ # Rust backend  
src/  
main.rs # Tauri entry, commands, tray  
config.rs # Config read/write (shared with MCP)  
database.rs # Connection pool, query execution  
mcp_server.rs # MCP protocol handler  
Cargo.toml  
tauri.conf.json # Tauri build config  
src/ # Frontend (web UI)  
index.html  
app.js # Connection management UI  
styles.css # Dark theme  
package.json
```

2.3 Config file format

Configuration is stored at `~/.config/pg-mcp/config.json` with `0600` permissions (owner-only read/write). The file contains credentials and must not be world-readable.

```
{
  "connections": [
    {
      "name": "shiftmatic-dev",
      "host": "localhost",
      "port": 5432,
      "database": "shiftmatic_dev",
      "user": "postgres",
      "password": "secret",
      "ssl": false,
      "readonly": true,
      "color": "#22c55e"
    }
  ],
  "activeConnection": "shiftmatic-dev",
  "uiPort": 5488
}
```

Each connection also supports a `connectionString` field. When set, it overrides the individual `host/port/database/user/password` fields. This is useful for managed databases (Railway, Supabase, Neon) that provide a single connection URI.

3. Active Connection Model

This is the most important design decision in pg-mcp. The user manually selects one active database. The AI agent can only query that database. The agent cannot switch databases or target a different connection. This is a hard gate, not a suggestion.

3.1 Core principle

The active connection model works like a physical safety switch. The user sets it before starting work, and the agent operates within that boundary. This prevents the most dangerous class of errors: an LLM confidently executing queries against the wrong database because it misinterpreted a prompt or hallucinated a connection name.

The MCP tools do **not** accept a `connection` parameter. There is no way for the agent to specify which database to use. Every tool call is automatically routed to whatever the user set as active.

3.2 Switching behavior

The user can switch the active connection in two ways:

- **Config UI** — Click the active connection card at the top of the UI, select a different connection from the dropdown.
- **CLI** — Run `pg-mcp activate <connection-name>` from the terminal.

When the active connection changes:

- The config file is updated with the new `activeConnection` value.
- The connection pool for the previous database is flushed and closed.
- The next MCP tool call reads the updated config and connects to the new database.
- The UI updates immediately: green border + "Live" badge moves to the new connection.

3.3 What the agent sees

The agent has access to a `list_connections` tool that returns all configured connections and indicates which one is active. This lets the agent say "you're currently connected to `shiftmatic-dev`, would you like to switch to `staging`?" — but it cannot perform the switch itself.

If the agent or a tool call attempts to reference a non-active connection:

Only the active connection can be queried.

Active: shiftmatic-dev (localhost:5432/shiftmatic_dev)

Available connections:

- shiftmatic-dev (active)
- shiftmatic-staging
- hospital-compliance

To switch, use the pg-mcp UI or run:

pg-mcp activate <connection-name>

4. Safety Model

Safety in pg-mcp is layered. Each layer is independent — a failure in one does not compromise the others.

4.1 Read-only enforcement

Every new connection defaults to **read-only mode**. This is enforced at two levels:

Layer	Mechanism	What it catches
Client-side	SQL keyword detection before query is sent	INSERT, UPDATE, DELETE, DROP, CREATE, ALTER, TRUNCATE, GRANT, REVOKE, COPY, VACUUM
Server-side	BEGIN TRANSACTION READ ONLY wrapping	Any write that bypasses keyword detection (e.g., writes inside functions, dynamic SQL)

To enable writes, the user must open the config UI, edit the specific connection, and toggle "Read-Only" off. This is deliberately a multi-step process in the UI — not a CLI flag, not an environment variable.

4.2 Write detection keywords

The client-side filter checks the first token of the normalized (uppercased, trimmed) SQL statement against this list:

```
INSERT, UPDATE, DELETE, DROP, CREATE, ALTER, TRUNCATE,  
GRANT, REVOKE, COPY, VACUUM, REINDEX, COMMENT, SECURITY,  
SET ROLE, RESET ROLE
```

This is a best-effort filter. The server-side READ ONLY transaction is the true enforcement layer. The client-side filter exists to provide faster, more descriptive error messages.

4.3 Connection isolation

- Each connection gets its own connection pool (max 5 connections per pool).
- Switching the active connection flushes the previous pool entirely.
- Pools have a 30-second idle timeout and 10-second connection timeout.
- Config file uses 0600 permissions — only the owning user can read credentials.
- Passwords are masked in all API responses from the config UI web server.

5. MCP Tools

5.1 Tool reference

All tools operate exclusively against the active connection. None accept a connection parameter.

list_connections

Parameters: (none)

Lists all configured connections. Indicates which is active and each connection's read/write mode. The agent can use this to suggest switching but cannot switch itself.

query

Parameters: sql: string

Executes a SQL query against the active database. Returns results as a formatted text table with column headers. Blocked by read-only enforcement if the query is a write operation.

list_tables

Parameters: schema?: string

Lists all tables and views grouped by schema. Defaults to all non-system schemas. Shows table name, type (table/view), and schema.

describe_table

Parameters: table: string

Returns column definitions (name, type, nullable, default), indexes, and foreign keys. Accepts schema-qualified names like "public.users".

list_schemas

Parameters: (none)

Lists all non-system schemas with their table and view counts.

test_connection

Parameters: (none)

Tests connectivity to the active database. Returns server version and latency in milliseconds.

get_table_sample

Parameters: table: string, limit?: number

Returns sample rows from a table (default 5, max 50). Useful for understanding data shape.

get_schema_diagram

Parameters: schema?: string

Generates a text-based entity-relationship diagram showing tables, columns (with PK markers), types, and foreign key relationships.

5.2 Response format

Every tool response begins with a connection banner. This is not optional — it appears on every single response, including errors. The banner uses box-drawing characters and emoji markers for maximum visibility in text-only contexts.

```

■ <package emoji> shiftmatic-dev
■ <link emoji> localhost:5432/shiftmatic_dev
■ <lock emoji> READ-ONLY

```

(query results or tool output follow)

Query results are formatted as text tables with column headers, separator lines, and row data. Results are capped at 100 rows with a truncation notice. Column values are capped at 60 characters.

5.3 Error handling

Errors are returned with `isError: true` in the MCP response. Error categories:

Error	Cause	User action
No active connection	No connection set as active	Open UI, activate a connection
Write blocked	Write query on read-only connection	Edit connection, disable read-only
Connection failed	Database unreachable or bad credentials	Check host/port/credentials in UI
Query error	Invalid SQL or permission denied	Fix the query (agent can retry)

6. Config UI

The config UI is a single-page application rendered in Tauri's native webview. It uses a dark theme designed for developer tooling contexts. The UI has four primary regions:

6.1 Active connection selector

The most prominent element on the page. Positioned at the top, below the header. Shows the currently active database with a green border and a "Live" badge. Clicking opens a dropdown of all configured connections.

Visual treatment:

- Green (#34d058) 2px border around the active card
- "Live" badge in green, positioned top-right of the card
- Pulsing green dot (8x8px) next to the connection name
- Connection name in monospace 16px bold
- Host/port/database in monospace 12px muted
- Chevron icon indicating the dropdown

Dropdown behavior:

- Opens below the active card, same width
- Lists all connections with color dot, name, host, and RO/RW badge
- Active connection has a checkmark and highlighted background
- Clicking a different connection immediately activates it
- Dropdown closes on selection or outside click

Subtext below selector:

"Your AI agent can only query this database. Switch here to change targets."

6.2 Connection cards

Below the active selector, an "All connections" section lists every configured connection as a card. Each card is a horizontal row with:

- **Color bar** — 6px wide vertical bar on the left in the connection's assigned color
- **Name** — Monospace 14px bold
- **Badges** — "Active" (green, if active), "Read-only" (blue) or "Read-write" (green)
- **Host info** — Monospace 11px muted, showing host:port/database and username

- **Actions** — Appear on hover: Activate (amber, only on non-active), Test, Edit, Delete (red)

Connection colors are auto-assigned from a palette of 8 colors (green, blue, amber, red, purple, cyan, orange, pink) to ensure visual distinction when scanning a list of 5+ connections.

6.3 Add/edit modal

A centered modal overlay with the following form fields:

Field	Type	Notes
Connection name	Text input	Required. Used as the identifier everywhere.
Connection string	Text input	Optional. Overrides host/port/database/user/password when set.
Host	Text input	Default: localhost
Port	Number input	Default: 5432
Database	Text input	Required if no connection string
User	Text input	Default: postgres
Password	Password input	Masked. Not prefilled on edit (keep existing).
SSL	Toggle	Default: off
Read-only	Toggle	Default: on. This is the safety default.

An "or" divider separates the connection string field from the individual fields. The layout uses a 2-column grid for Host+Port and User+Password rows to keep the modal compact.

The footer has three buttons: **Test Connection** (runs a connectivity check and displays results inline below the toggles), **Cancel**, and **Save** (amber primary button).

6.4 Agent setup panel

A collapsible details panel at the bottom of the page showing copy-pasteable configuration snippets for Claude Desktop, Claude Code, and other MCP clients. Each snippet has a copy button.

7. CLI Interface

The same Tauri binary supports CLI commands for users who prefer the terminal. These are especially important for the `activate` command, which lets users switch the active database without opening the UI.

Command	Description
<code>pg-mcp</code>	Start the MCP server (stdio transport). This is what AI agents invoke.
<code>pg-mcp ui</code>	Launch the Tauri app with the config UI.
<code>pg-mcp activate <name></code>	Switch the active connection. Writes to config, flushes pool. Same effect as clicking in the UI.
<code>pg-mcp test [name]</code>	Test connectivity. Defaults to active connection if no name given.
<code>pg-mcp list</code>	List all configured connections with active indicator.
<code>pg-mcp help</code>	Print usage and setup instructions.

Note: The `activate` command is the only way to switch databases from the terminal. The agent process cannot call it — it requires direct user execution. This maintains the hard-gate safety model.

8. Visual Design

The UI uses a dark theme designed for developer tooling. The aesthetic is flat, functional, and information-dense without being cluttered.

8.1 Color system

Token	Hex	Usage
bg-root	#0a0a0b	Page background
bg-surface	#141416	Cards, panels
bg-elevated	#1c1c1f	Buttons, hover states
border	#2a2a2e	All borders
text-primary	#e8e8ec	Main text, headings
text-secondary	#8e8e96	Labels, secondary info
text-muted	#5c5c64	Placeholders, hints
accent	#f0a030	Primary actions, active states, amber accent
green	#34d058	Active connection, success, live indicator
blue	#58a6ff	Read-only badge
red	#f85149	Delete actions, errors

Semantic colors (green, blue, red) are used with a dim background variant at ~12% opacity for badges and status indicators.

8.2 Typography

Context	Font	Weight	Size
App title	JetBrains Mono	600	20px
Connection name	JetBrains Mono	600	14-16px
Host/port info	JetBrains Mono	400	11-12px
Form inputs	JetBrains Mono	400	13px

Buttons, labels	Outfit	500	12-13px
Body text, descriptions	Outfit	400	12-13px
Badges	System sans	600	10px

JetBrains Mono is used for anything that represents database or code content (connection names, hosts, SQL). Outfit is used for UI chrome (buttons, labels, descriptions). This creates a clear visual hierarchy between "data" and "interface".

8.3 Component tokens

- **Border radius:** 8px (small elements), 10-12px (cards, modal)
- **Borders:** 1px solid #2a2a2e everywhere. 2px solid #34d058 for active connection only.
- **Card padding:** 16-20px horizontal, 16-20px vertical
- **Button padding:** 8px 16px (normal), 5px 10px (small)
- **Connection color bar:** 6px wide, 44-48px tall, 3px border-radius
- **Hover actions:** Card actions hidden by default, shown on hover (opacity transition 0.15s)
- **Toggle switches:** 40x22px track, 16x16px thumb, amber accent when on
- **Modal overlay:** rgba(0,0,0,0.6) with 4px backdrop blur

9. Distribution & Installation

pg-mcp is distributed as a Tauri application, producing native installers for each platform.

Platform	Format	Size (est.)	Notes
macOS	.dmg / .app	~5-8 MB	Universal binary (Intel + Apple Silicon)
Windows	.msi / .exe	~5-8 MB	Requires WebView2 (pre-installed on Win 11)
Linux	.deb / .Applmage	~5-10 MB	Requires WebKitGTK

Installation flow:

- 1. Download the installer for your platform from GitHub Releases (or Homebrew on macOS).
- 2. Run the installer. The pg-mcp binary is placed in your PATH.
- 3. Run `pg-mcp ui` to open the config interface.
- 4. Add your database connections and activate one.
- 5. Add pg-mcp to your AI agent's MCP config (copy-paste from the setup panel).

Agent-side config (Claude Desktop example):

```
{
  "mcpServers": {
    "postgres": {
      "command": "pg-mcp",
      "args": ["serve"]
    }
  }
}
```

Claude Code:

```
claude mcp add postgres -- pg-mcp serve
```

Note that the agent runs `pg-mcp serve`, not `pg-mcp ui`. The `serve` command starts the MCP server in stdio mode with no UI. The UI is a separate process the user launches independently.

10. Future Considerations

These are not committed features. They represent directions worth exploring after the v1.0 core is stable.

Environment variable interpolation

Support `$DATABASE_URL` and similar env var references in connection strings. Useful for 12-factor setups where credentials come from the environment rather than a config file. The config file would store `"connectionString": "$SHIFTMATIC_DB_URL"` and resolve at connection time.

SSE/HTTP transport

Add an optional SSE transport alongside stdio for MCP clients that prefer HTTP. The Tauri app could host this on a localhost port, similar to how the config UI works today. This would expand compatibility to web-based agents.

Query history

Store a local log of queries executed through pg-mcp, tagged by connection and timestamp. Useful for auditing what the agent did during a session. Could be displayed in the UI as a collapsible history panel.

System tray integration

Tauri supports system tray icons natively. pg-mcp could show the active connection name and color in the tray, with a quick-switch menu for changing databases without opening the full UI window.

Connection groups / projects

Group connections by project (e.g., "ShiftMatic" has dev/staging/prod, "Hospital" has compliance/analytics). The UI would show a project-level grouping with collapsible sections.

Schema diffing

A tool that compares schemas between two connections (e.g., dev vs. staging). This requires temporarily reading from a non-active connection, which would need careful UX — perhaps a dedicated "Compare" mode in the UI that briefly opens a read-only connection to the second database.

MySQL / SQLite support

The architecture could generalize to other database engines. The connection config would gain a type field (postgres, mysql, sqlite), and the database layer would use the appropriate driver. The MCP tools would remain the same — just the underlying queries would differ.

End of specification. For questions or feedback, open an issue on the GitHub repository.